



AI61002: Deep Learning Foundations and Applications

Module 4: Regularization Techniques for Neural Networks

Dr. Somdyuti Paul

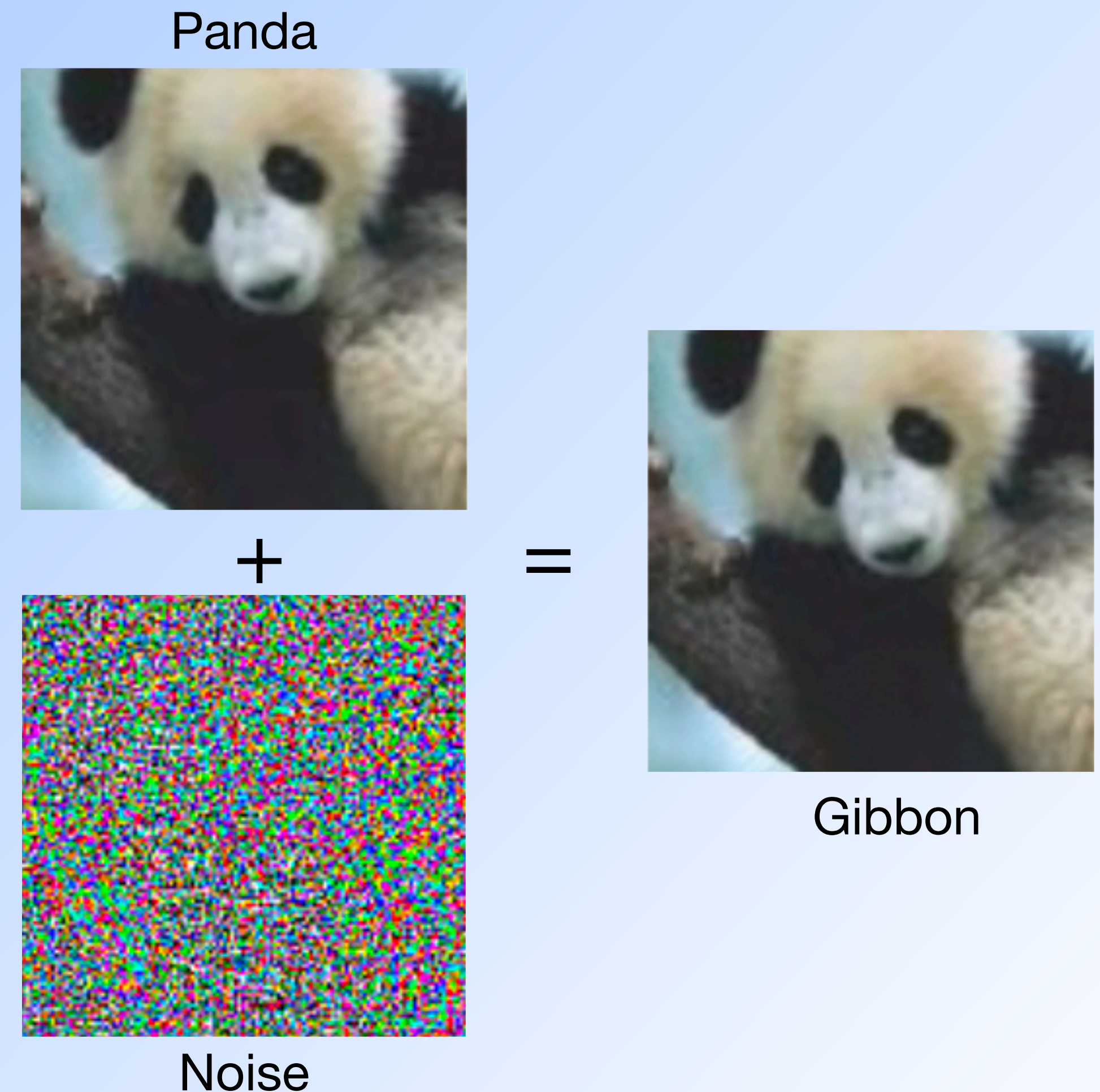
Regularization for Deep Neural Networks

- Regularization trades off variance for increased bias to control model overfitting.
- A regularized deep neural network with more trainable parameters generally performs better than an unregularized network having fewer parameters.
- In addition to general regularization techniques that are applicable to machine learning models, some special kind of regularization techniques could be applied to deep neural networks.

Traditional Regularization Techniques	Regularization Techniques Specific to Deep Learning
• Parameter norm penalties • Data augmentation • Early stopping • Parameter sharing • Ensemble methods	• Dropout • Batch Normalization

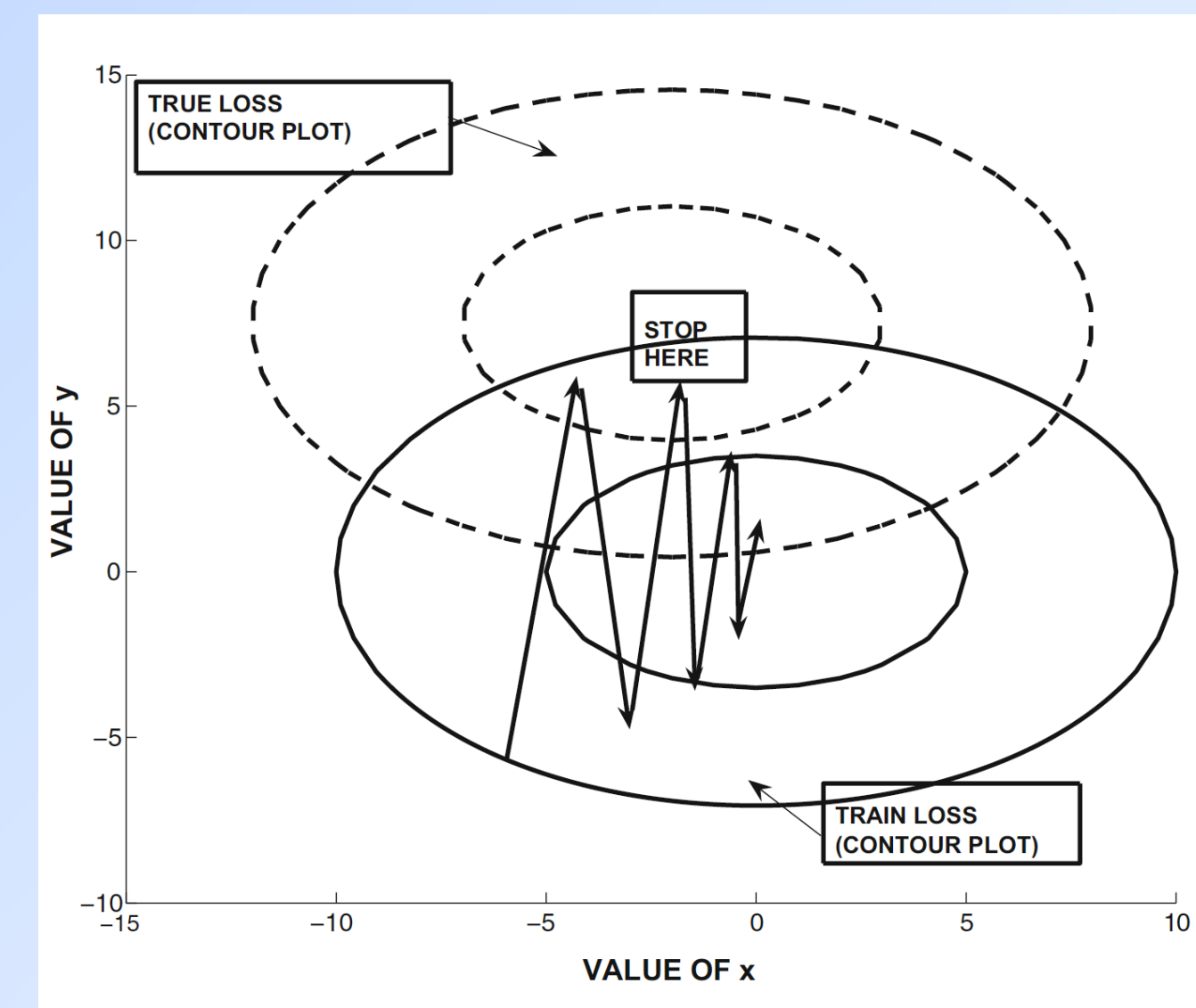
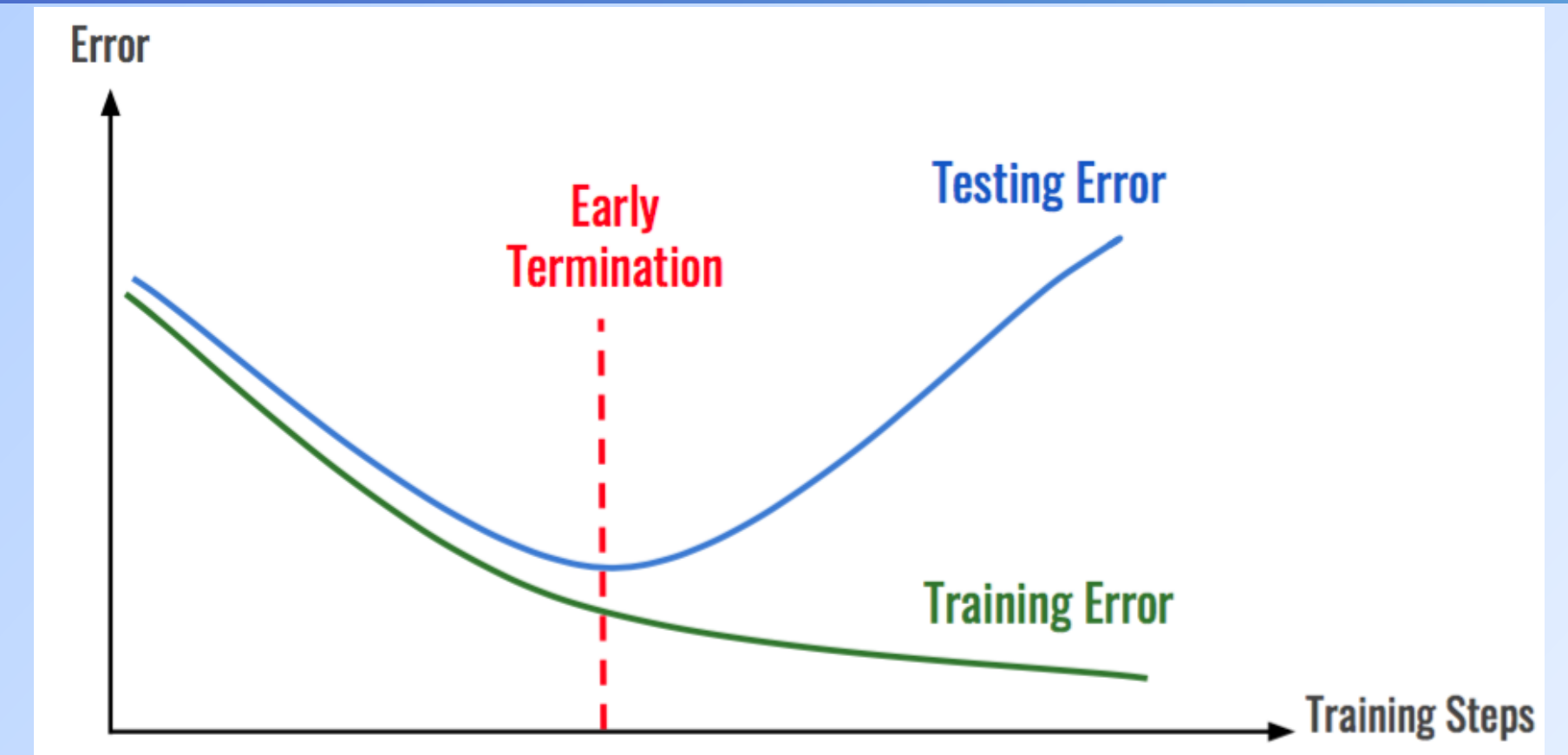
Data Augmentation

- Adding more data to the training set is an effective way to attain better regularization.
- In addition to acquiring more data, the size of the training set can be increased in the following ways:
 - Applying transformations to existing data
 - Generating synthetic data
 - Adding controlled noise to data.
- It must be ensured that the labels are invariant to the transformations applied for the purpose of augmentation.



Early Stopping

- Early stopping refers to terminating the training process when validation loss stops showing improvement.
- Model weights that correspond to the smallest validation loss during training need to be saved.
- This simple scheme acts as a constraint on the optimization process that restricts the distance of the final solution from the initialization point, which introduces regularization.
- The intuition behind early stopping is that since the true cost function is unknown and likely to be different from the cost function constructed using the training samples, better solutions with respect to the true cost function might lie further away from the best solution with respect to the training loss.



Parameter Sharing

- Sharing the same parameters across multiple connections reduces the number of learnable parameters of the model, thereby introducing regularization.
- Parameter sharing makes sense when the functions computed at different nodes ought to be same or very similar.
- Examples of parameter sharing:
 - convolutional neural networks - exploits translational invariance of visual data to apply the same convolution filters to multiple spatial locations.
 - Recurrent neural networks - shares parameters across different timestamps
 - Autoencoders - could be designed to share symmetric weights between encoder and decoder.

Norm Penalties

- An effective way of adding regularization to parameterized models is to add norm penalties to the cost function as follows:

$$\tilde{J}(\theta) = J(\theta) + \lambda \Omega(\theta)$$

Where $J(\theta)$ is the cost function computed using the model's loss, $\Omega(\theta)$ are parameter norms, and $\lambda > 0$ controls the relative importance of the norm penalty term.

- The regularized cost function $\tilde{J}(\theta)$ is subsequently minimized by the optimization algorithm.
- The norm penalty adds regularization by constraining the value of the parameters, thereby lessening their influence on the solution.
- Typically in neural networks, weights of the hidden layers are regularized as above, while biases are unregularized.

Norm Penalties

L_2 Regularization

L_2 or *ridge* regularization uses L_2 norm as the norm penalty for the parameters.

The regularized cost function in this case is given by:

$$\tilde{J}(\mathbf{w}, \mathbf{b}) = J(\mathbf{w}, \mathbf{b}) + \frac{\lambda}{2} \mathbf{w}^T \mathbf{w}$$

Taking partial derivative with respect to w_i :

$$\frac{\partial \tilde{J}(\mathbf{w}, \mathbf{b})}{\partial w_i} = \frac{\partial J(\mathbf{w}, \mathbf{b})}{\partial w_i} + \lambda w_i$$

So, the parameter update rule is:

$$w_i \leftarrow (1 - \alpha\lambda)w_i - \alpha \frac{\partial J(\mathbf{w}, \mathbf{b})}{\partial w_i}$$

Thus, the weights are attenuated by a factor of $(1 - \alpha\lambda)$ at each step in addition to being updated according to the gradient of the cost function.

Norm Penalties

L_1 Regularization

In case of L_1 or Lasso regularization, the norm penalty is applied using L_1 norm, and the associated regularized cost function is expressed as follows:

$$\tilde{J}(\mathbf{w}, \mathbf{b}) = J(\mathbf{w}, \mathbf{b}) + \lambda \|\mathbf{w}\|_1$$

Taking partial derivative with respect to w_i :

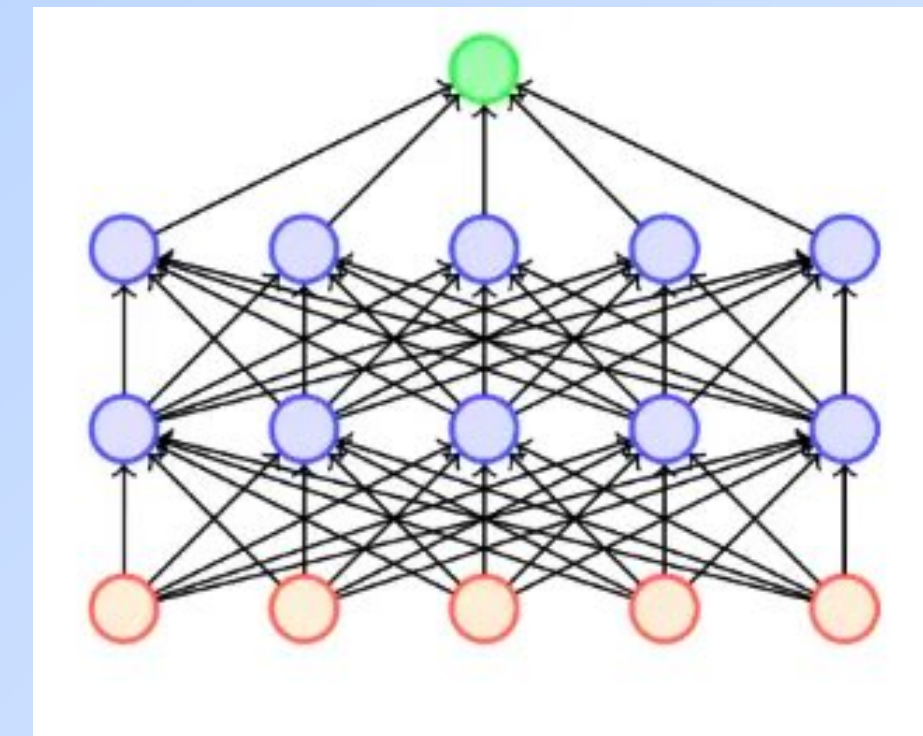
$$\frac{\partial \tilde{J}(\mathbf{w}, \mathbf{b})}{\partial w_i} = \frac{\partial J(\mathbf{w}, \mathbf{b})}{\partial w_i} + \lambda \text{sign}(w_i)$$

$$w_i \leftarrow w_i - \alpha \lambda \text{sign}(w_i) - \alpha \frac{\partial J(\mathbf{w}, \mathbf{b})}{\partial w_i}$$

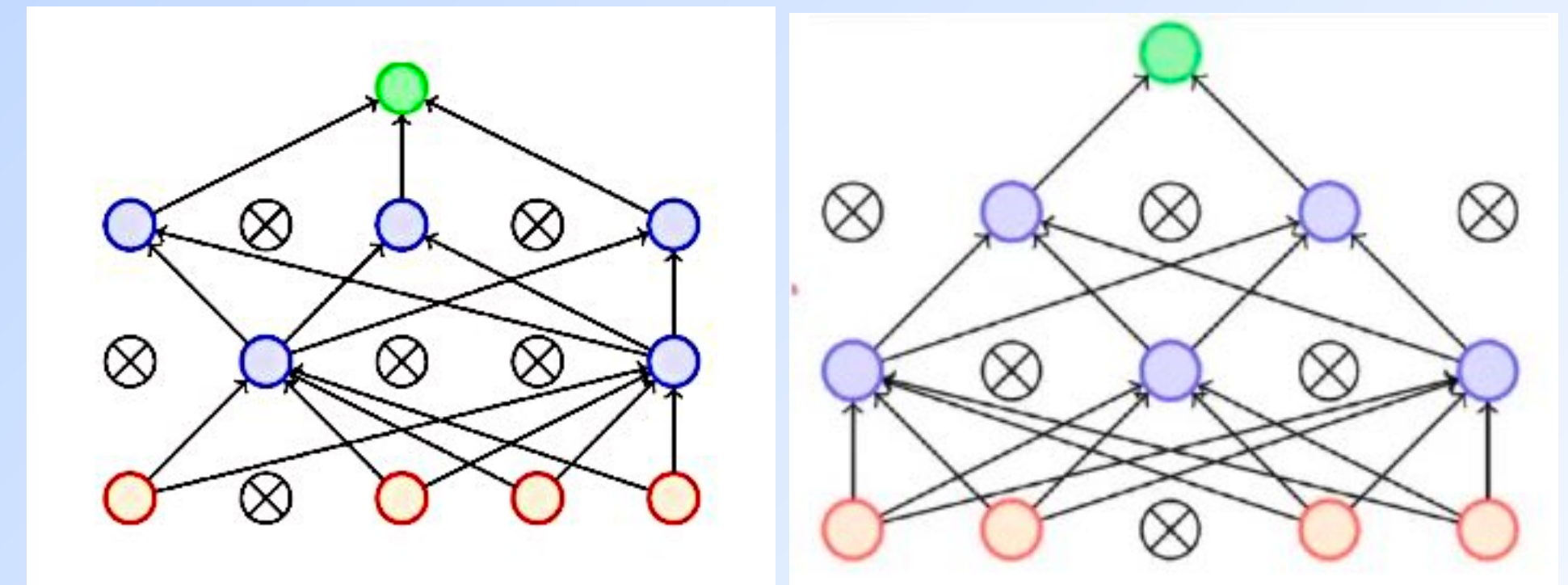
Thus, L_1 regularization also tends to shift the weights towards 0. However, unlike L_2 regularization, in L_1 regularization many of the learned weights could be exactly 0, leading to sparse solutions allowing for automatic feature selection.

Dropout

- Dropout in neural networks refers to deactivating a randomly chosen subset of hidden units from a base network while performing prediction ([Srivastava et al., 2014](#)).
- The n^{th} hidden unit is dropped with probability p_n .
- It provides an effective means of regularization by implicitly creating an ensemble of models in a computationally inexpensive fashion.
- Thus, while using dropout, virtually a different model is used for prediction on each minibatch.
- Thus, for a neural network having M hidden units, 2^M different models with shared weights could be used for prediction by employing dropout.



Base neural network



Two networks after applying dropout

Dropout

- Dropout prevents feature co-adaptation by encouraging smaller subset of features to have predictive power.
- By deactivating randomly chosen hidden units, dropout enforces redundancy between learned features at different units which leads to robustness.
- Dropout is not applied during inference to utilize the full capacity of the neural networks.
- However, the activations need to be scaled to account for the different expected number of activations during training and inference:
 - *Strategy 1 - scaling during inference*
scale the activation of the n^{th} hidden unit by $1 - p_n$ during inference
 - *Strategy 2 - scaling during training*
the activation of the n^{th} hidden unit by $\frac{1}{1 - p_n}$ during training
- Generally, dropout and batch-normalization are not applied together, as variance shift due to dropout causes unstable behavior during inference when applied together with batch normalization. (Li et al., 2019).